

AI-Assisted Coding Project in Python

Band Gap Prediction of Non-Metals Using Machine Learning

Muhammad Talha Nasir
CMS@TUBAF matr.no. 71093

July 11, 2025

Abstract

Accurate prediction of the electronic band gap (E_g) of non-metallic materials is critical for semiconductor and optoelectronic design. Traditional density functional theory (DFT) methods often underestimate band gap E_g and also there is an issue of cost constraint. Therefore a better and cheaper way of predicting Band Gap is using Machine Learning Models. Here, we develop a two-stage **XG-Boost**-based machine learning workflow first classifying metals vs. non-metals, then predicting Band Gap E_g for non-metals using 132 compositional descriptors. The dataset used for training the XG-Boost model was, 4604 experimentally measured band gaps. We achieve a classification accuracy of 93.8% (AUC 0.982) and regression performance of RMSE = 0.630 eV, $R^2 = 0.809$, MAD = 0.695 outperforming PBE DFT and matching more expensive hybrid approaches.

Keywords: ChatGPT, Band Gap, Machine Learning, XGBoost, Material Property Prediction

1 Introduction

Understanding and predicting the electronic properties of materials is a fundamental goal in materials science, especially for applications in semiconductors, photovoltaics, and optoelectronic devices. Among these properties, the band gap plays a central role in determining whether a material behaves as a conductor, insulator, or semiconductor. Traditional approaches to estimating such properties ranging from density functional theory (DFT) to laboratory measurements can be accurate but are often time-consuming, computationally expensive, or experimentally intensive.

This study focuses on using XGBoost to predict the band gap of Non-metals. We use a dataset from matbench, containing metal and non-metal compositions with experimentally determined band gaps.

1.1 Band gap

The band gap (E_g) is a crucial material property that determines the electronic behavior of materials. It is defined as an energy range within a solid material where no electron states can exist. Essentially, the band gap represents the energy required to move an electron from the valence band to the conduction band [1].

1.2 Overview of the application of machine learning in materials science

Machine learning is widely used in materials science and demonstrates superiority in both time efficiency and prediction accuracy. As shown in Fig. 1, the applications of machine learning in materials discovery and design can be divided into three main classes: material property prediction, new materials discovery and various other purposes. In research on material property prediction, regression analysis methods are typically used, and both macroscopic and microscopic properties can be predicted. The main idea underlying the application of machine learning in new materials discovery is to use a probabilistic model to screen various combinations of structures and components and finally to select a material with good performance from the candidate set by means of density functional theory (DFT)-based validation. In addition, machine learning is also used for other purposes in materials science, such as process optimization and density function approximation. [2]. In this study we are focusing on Material Property prediction.

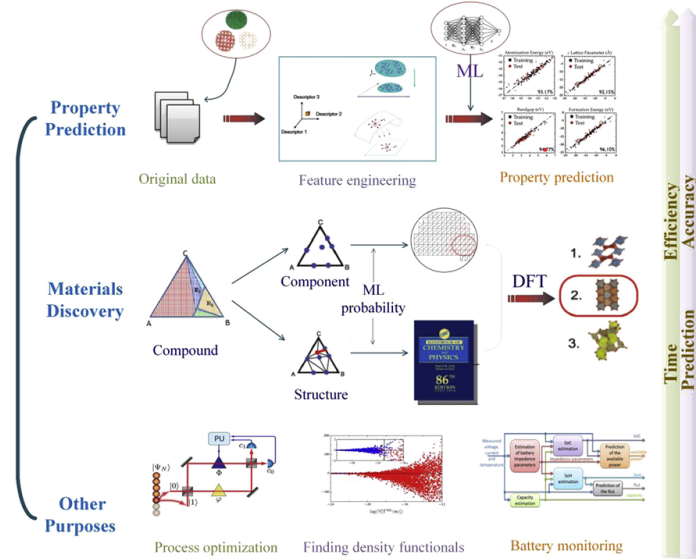


Figure 1: An overview of application of machine learning in Material Science.

1.3 XGBoost

XGBoost, short for Extreme Gradient Boosting, is a highly advanced machine learning algorithm known for its strong performance in both classification and regression tasks. It builds an ensemble of decision trees through an iterative process, where each new tree aims to correct the errors made by the previous ones. By using gradient descent to optimise this process, XGBoost efficiently reduces prediction errors, continuously improving the models accuracy by fine-tuning the trees parameters. It has several advantages, including handling missing values, preventing overfitting through regularization, and providing feature importance scores. Due to these characteristics, XGBoost is particularly suitable for predicting material properties like the band gap, where complex relationships exist between the input features and the output properties.

1.4 Data and feature engineering

Our dataset is taken from matbench [3] and comprises:

- **Data:** Consists of 4,604 compositions of metals and non-metals.
- **Labels:** A total of 4,604 experimentally calculated band gaps. The dataset includes 2,154 experimentally measured Band gap E_g values for non-metals and 2,450 for metals.
- **Features:** 132 compositional features (sum, difference, max, min of elemental properties) extracted from open databases (atomic number, radii, electronegativity scales, thermochemical data, etc.)

Data were split 80/20 for training/testing, stratified to balance metals vs. non-metals.

1.5 Machine learning model

We employ a two-stage XGBoost workflow:

1. **Classification:** XGBoost classifier predicts metal ($E_g = 0$) vs. non-metal ($E_g > 0$). Trained on 4,604 experimentally calculated band gaps
2. **Regression:** XGBoost regressor predicts E_g for compositions classified as non-metals. Trained on 2,154 experimentally calculated band gaps of Non-metals.

2 Prompt

Heres a short summary of what we want our code to do. The code is divided into two python file one for training the XGBoost classification and regression model, second one for testing the trained model on unseen data. First we extract elemental properties like (ionic radius, atomic number etc) from the

composition (LiMgSnPd,InAs etc) using pymatgen library then we use these elemental properties to train the XGBoost model. After training we save the classification and regression models. In the second python file we give the XGBoost classification and regression model, unseen Non-metals compositions whose experimental and DFT calculated band gap we already know. The band gap predicted by our model is compared with the result of SVM model.

PROMPTS 1

Think like a material Scientist. Provide me the code.

TASK

I am training a machine learning model to first classify Non-Metals from Metals and then predict the bandgap of the Non-Metals.

PROCEDURE

For Dataset use `matbench_expt_gap` dataset from the `matminer` library. In the dataset convert composition to Composition objects using `pymatgen` library composition. Using the `pymatgen` composition, calculate the properties of the composition using `matminer` Element Properties. Now split the dataset into an 80/20 ratio keeping in mind that both the train and test dataset have an equal number of metals and nonmetals. Now classify the Non-Metals=1 and Metals=0 using the XGBoost classification model. For evaluation of the Classification model, use accuracy, ROC-AUC, precision, classification report. Also, calculate the confusion matrix (non-metal = positive) and plot it using `seaborn`. Now, using only the Non-metals dataset, train a regression model using XGBoost that predicts the bandgap of composition. For evaluation of the Regression model, use RMSE, R, and MAD. Now save the classification and regression model.

PLOT

Plot the dataset in `matbench_expt_gap`. Predicted band gap $E_{g,XGB}$ versus measured band gap $E_{g,exp}$ for 921 compounds in the test set.

Figure 2: Snippet of prompt for training XGBoost model for classification and regression

PROMPT 2

TESTING MODEL

Continue with the previous code/keep the previous output as a starting point. Now do the following things:

1. Create a dataframe for formulas ['GaN', 'CdTe', 'LiF', 'TiO2', 'CuSbS2', 'ZnS', 'Cu2ZnSnS4', 'PbTe', 'GaAs', 'ZnO'] and its actual band gap Eg_actual = [3.2, 1.6, 14.2, 3.42, 1.38, 3.91, 1.6, 0.19, 1.52, 3.44].
2. Convert formula to Composition object.
3. Featurize the Composition.
4. Classify whether it is a metal or non-metal.
5. Find the band gap of the formulas classified as non-metal.
6. For model evaluation, calculate RMSE, MAD, and R.
7. Now, find the deviation of the actual band gap from the predicted band gap in percentage like $3.41(-1\%)$.

Figure 3: Snippet of prompt for testing the model on unseen dataset.

3 Code Listing

```
1 import pandas as pd
2 from matminer.datasets import load_dataset
3 from pymatgen.core.composition import Composition
4 from matminer.featurizers.composition import ElementProperty
5 from sklearn.model_selection import train_test_split
6
7 import xgboost as xgb
8 from sklearn.metrics import (
9     accuracy_score, confusion_matrix, mean_absolute_error,
10     roc_auc_score, classification_report, mean_squared_error, r2_score,
11 )
12 import seaborn as sns
13 import matplotlib.pyplot as plt
14 from xgboost import XGBRegressor
15 import numpy as np
16 import joblib
17
18 if __name__ == "__main__":
19
20     # load datasets
21     df = load_dataset("matbench_expt_gap")
22
23     # Convert formulas to Composition objects
24     df['composition'] = df["composition"].apply(Composition)
25
26     # Instantiate the featurizer
27     ep = ElementProperty.from_preset("magpie")
28
29     # Generating the feature matrix
30     feat_array = ep.featurize_many(df["composition"], ignore_errors=True)
31     feat_df = pd.DataFrame(feat_array,
32                           columns = ep.feature_labels(),
33                           index = df.index)
34     df = pd.concat([df, feat_df], axis=1)
35
36     # renaming "gap expt" to "Eg" and setting it as index
```

```

37 df.rename(columns={"gap_expt": "Eg"}, inplace=True)
38 df.set_index('composition', inplace=True)
39
40 # Splitting dataset
41 x = df.drop('Eg', axis=1)
42 y = df['Eg']
43
44 x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
↳ random_state=42, stratify=y==0) # keeps metal/non metal ratio the same in both
↳ sets but on y it is not working correctly
45
46 # Label metals (0) vs. non-metals (1) based on experimental band gap
47 y_train_clf = y_train.apply(lambda x: 0 if x==0 else 1)
48 y_test_clf = y_test.apply(lambda x: 0 if x==0 else 1)
49
50 # Train XGBoost classifier
51 model = xgb.XGBClassifier(eval_metric='logloss', random_state=42)
52 model.fit(x_train, y_train_clf)
53
54 # Predict
55 y_pred_clf = model.predict(x_test)
56 y_proba = model.predict_proba(x_test)[: , 1] # Probabilities for ROC-AUC
57
58 # Evaluate classifier
59 print("    Evaluation Metrics:")
60 print("Accuracy:", accuracy_score(y_test_clf, y_pred_clf))
61 print("ROC-AUC:", roc_auc_score(y_test_clf, y_proba))
62
63 # Optional: More detailed report
64 print("\nClassification Report:\n", classification_report(y_test_clf, y_pred_clf,
↳ target_names=['Non-metal', 'Metal']))
65
66 # Get confusion matrix
67 cm = confusion_matrix(y_test_clf, y_pred_clf)
68
69 # Label positions and annotations
70 labels = np.array([["TP", "FP"], ["FN", "TN"]])
71 counts = cm
72
73 # Create a string matrix with count and label
74 annot = np.empty_like(cm).astype(str)
75 for i in range(cm.shape[0]):
76     for j in range(cm.shape[1]):
77         annot[i, j] = f"{cm[i, j]}\n{labels[i, j]}"
78
79 # Plot
80 plt.figure(figsize=(5, 4))
81 sns.heatmap(cm, annot=annot, fmt="", cmap="Reds", cbar=False,
82             xticklabels=["non-metal", "metal"],
83             yticklabels=["non-metal", "metal"])
84 plt.xlabel("actual")
85 plt.ylabel("predicted")
86 plt.title("(c)")
87 plt.tight_layout()
88 plt.show()
89
90 # Regression on non-metals only
91
92 # Dataset used for regression (Only non-metals)
93 x_reg = df[df['Eg']>0].drop('Eg', axis=1)
94 y_reg = df[df['Eg']>0]['Eg']
95
96 # splitting the data
97 x_train_reg, x_test_reg, y_train_reg, y_test_reg = train_test_split(x_reg, y_reg,
↳ test_size=0.2, random_state=42)
98
99 # Training Regression model
100 reg = XGBRegressor(random_state=42)
101 reg.fit(x_train_reg, y_train_reg)
102
103 # predicting
104 y_pred_reg = reg.predict(x_test_reg)
105
106 # Evaluating Regression Model
107 print("    Evaluating Regression Model:")

```

```

108 rmse = np.sqrt(mean_squared_error(y_test_reg, y_pred_reg))
109 r2 = r2_score(y_test_reg, y_pred_reg)
110 mad = mean_absolute_error(y_test_reg, y_pred_reg)
111
112 print(f"RMSE [eV]: {rmse:.3f}")
113 print(f"R Score : {r2:.3f}")
114 print(f"MAD: {mad:.3f}")
115
116 # Saving the models
117 joblib.dump(model, 'XGBClassifier.joblib')
118 joblib.dump(reg, 'XGBRegressor.joblib')
119
120 # --- Plot experimental test set with ideal and fit lines ---
121
122 # Scatter plot
123 plt.figure(figsize=(6,6))
124 plt.scatter(y_test_reg, y_pred_reg, alpha=0.6, label='Predictions')
125
126 # Ideal line (y = x)
127 lims = [min(y_test_reg.min(), y_pred_reg.min()), max(y_test_reg.max(), y_pred_reg.
↪ max())]
128 plt.plot(lims, lims, 'r--', label='Ideal (y = x)')
129
130 # Labels and legend
131 plt.xlabel('Measured $E_g$ (exp)')
132 plt.ylabel('Predicted $E_g$ (XGB)')
133 plt.title('Predicted vs Measured (Experimental)')
134 plt.legend()
135 plt.tight_layout()
136 plt.show()

```

Code Listing 1: Output by ChatGPT for 1st prompt, BandGap_Model_Training.py

```

1 import pandas as pd
2 from pymatgen.core.composition import Composition
3 from matminer.featurizers.composition import ElementProperty
4
5 from sklearn.metrics import (
6     mean_absolute_error, mean_squared_error, r2_score
7 )
8 import numpy as np
9 import joblib
10
11 if __name__ == "__main__":
12
13     # Loading the saved classification and regression model
14
15     XGBclf = joblib.load('XGBClassifier.joblib')
16     XGBreg = joblib.load('XGBRegressor.joblib')
17
18     # --- Test on new formulas ---
19     print("Testing custom formulas...")
20     test_formulas = ['GaN', 'CdTe', 'LiF', 'TiO2', 'CuSbS2', 'ZnS',
21                     'Cu2ZnSnS4', 'PbTe', 'GaAs', 'ZnO']
22     Eg_actual = [3.2, 1.6, 14.2, 3.42, 1.38, 3.91, 1.6, 0.19, 1.52, 3.44]
23     df_test = pd.DataFrame({"formula": test_formulas, "Eg_actual": Eg_actual})
24
25     # Instantiate the featurizer
26     ep = ElementProperty.from_preset("magpie")
27     df_test["Composition"] = df_test["formula"].apply(lambda x: Composition(x))
28     df_test = ep.featurize_dataframe(df_test, col_id="Composition", ignore_errors=True)
29     features = ep.feature_labels()
30     df_test = df_test.dropna(subset=features)
31
32     # Classify and predict
33     print("Classifying and predicting band gaps...")
34     df_test["label"] = XGBclf.predict(df_test[features])
35     df_pred_nm = df_test[df_test["label"]!=1].copy()
36     df_pred_nm["Eg_pred"] = XGBreg.predict(df_pred_nm[features])
37
38     # Metrics
39     rmse_new = np.sqrt(mean_squared_error(df_pred_nm["Eg_actual"], df_pred_nm["Eg_pred"]
↪ ))
40     r2_new = r2_score(df_pred_nm["Eg_actual"], df_pred_nm["Eg_pred"])
41     mad_new = mean_absolute_error(df_pred_nm["Eg_actual"], df_pred_nm["Eg_pred"])

```

```

42 print(f"Custom Test RMSE: {rmse_new:.3f}, R2: {r2_new:.3f}, MAD: {mad_new:.3f}")
43
44 df_pred_nm["Deviation_%"] = (df_pred_nm["Eg_pred"]-df_pred_nm["Eg_actual"])/
↪ df_pred_nm["Eg_actual"]*100
45 df_pred_nm["Eg_pred_str"] = df_pred_nm.apply(
46     lambda r: f"{r['Eg_pred']:.2f} ({r['Deviation_%']:+.0f}%)", axis=1)
47 print(df_pred_nm[["formula", "Eg_actual", "Eg_pred_str"]].to_string(index=False))

```

Code Listing 2: Output by ChatGPT for 2nd prompt ,BandGap_Model_Testing.py

The output of the ChatGPT does not guarantee that the code generated will work without any flaws. These flaws are discussed in Sec. 5.

4 Tests for Verification

The XGBoost models demonstrated good performance in both classification and regression tasks related to experimental band gap prediction. In the follow section we will see the performance of Classification and Regression Model. We will also compare the model performance with the SVM model trained in zhuo-et-al research paper.

4.1 Model Performance

Classification Evaluation: The classifier was trained to distinguish between metallic ($E_g = 0$) and non-metallic ($E_g > 0$) materials. It achieved a high **accuracy of 0.938** and a **ROC-AUC of 0.983**, indicating excellent discriminative power. The confusion matrix Fig. 4 confirms this performance with 451 true positives and only 18 false negatives, suggesting that most non-metals were correctly identified. It is worth noting that the false-positives (FP) are slightly higher than false-negatives (FN), 39 compared with 18, meaning that there is a slight preference to mistakenly classify a metal as a nonmetal. The F1-scores for both classes (0.94) demonstrate a balanced trade-off between precision and recall.

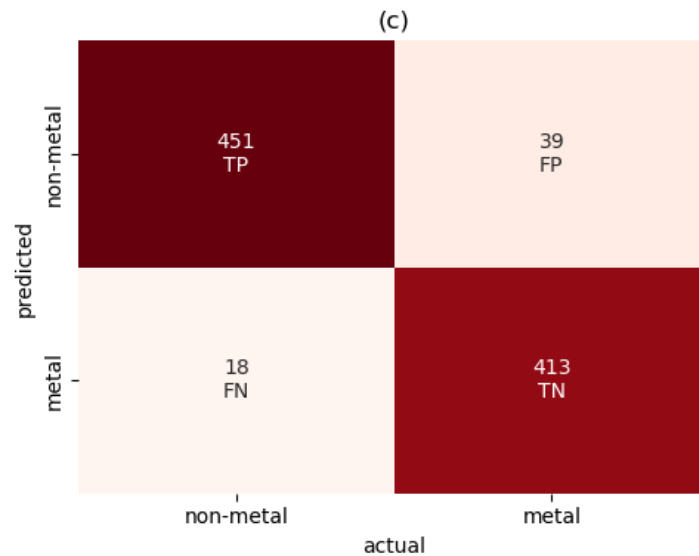


Figure 4: Confusion matrix showing the performance of the classification model.

Table 1: Classification report showing precision, recall, and F1-score for Non-metal and Metal classes.

Class	Precision	Recall	F1-Score
Non-metal	0.96	0.92	0.94
Metal	0.91	0.96	0.94
Accuracy	0.94		
Macro avg	0.94	0.94	0.94
Weighted avg	0.94	0.94	0.94

Regression Evaluation: For non-metallic materials, the XGBoost regressor predicted band gaps with a **Root Mean Squared Error (RMSE) of 0.630 eV**, an **R^2 score of 0.809**, and a **Mean Absolute Deviation (MAD) of 0.384 eV**. These results indicate that model performs a good job in predicting the Band Gap which can be seen in Table. 3 where we compare our results with results from SVR Model.

Table 2: Evaluation metrics for the regression model performance.

Metric	Value
RMSE [eV]	0.630
R^2 Score	0.809
MAD	0.384

The classifier effectively filters metals from non-metals, while the regressor provides an accurate estimate of band gap values.

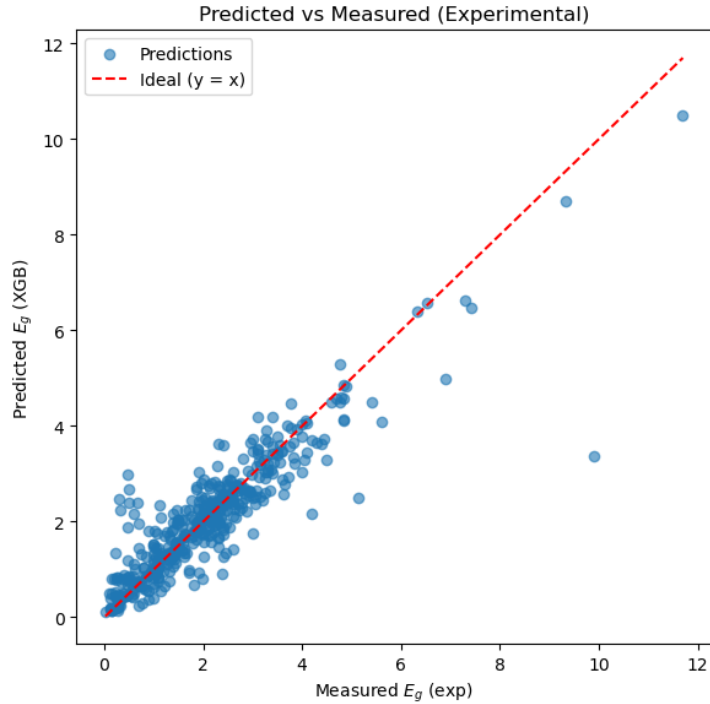


Figure 5: Predicted vs. measured band gaps for non-metallic compounds using the XGBoost regressor.

Each point in Fig. 5 represents a material. The red dashed line indicates the ideal prediction line ($y = x$). The plot shows that the model captures the overall trend accurately, though slight underestimation is observed for wide band-gap materials

4.2 Test: Comparing XGBoost Performance with SVM Model from Zhuo *et al.*[4]

To benchmark the performance of our composition-based XGBoost pipeline, we compare it against the model proposed by Zhuo *et al.* (2018), which used a Support Vector Machine (SVM) for classification and Support Vector Regression (SVR) for band gap prediction using experimentally measured data.

Table 3: Comparison of our XGBoost model with the SVM model from Zhuo *et al.* [4]

Metric	Our XGBoost Model	Zhuo <i>et al.</i> SVM Model
Classification Accuracy	93.8%	92.0%
Classification ROC-AUC	0.983	0.970
Regression RMSE (eV)	0.630	0.450
Regression R ² Score	0.809	0.90
Training Set	~4604 samples	3896 samples
Regression Target	Non-metals only	Non-metals only

Key observation:

Classification: The XGBoost classifier slightly outperforms the SVC model with a 2% higher accuracy and ROC-AUC, suggesting better discrimination between metals and non-metals.

Regression: Zhuos SVR model achieves a lower RMSE (0.45 eV) and higher R (0.90) than our XGBoost regressor (RMSE: 0.63 eV, R: 0.81), indicating better regression performance

To quantitatively compare the capability of the XGBosst Regression machine-learning model, we compare E_g for 10 specific compounds, unseen by our algorithm, that have also been examined at many different levels of theory including using the PBE functional, the hybrid functional (HSE), the GW (fully-self-consistent)method, and a PBE-trained machine learning model reported inAFLOW-ML.[4]

Table 4: Comparison of Experimentally Measured Band-Gap Values, with the Band Gaps Calculated Using the PBE Functional, the GW Approach, the AFLOW-ML Project, and comparison of SVR and XGBoost Model.

composition	$E_{g,exp}$	$E_{g,PBE}$	$E_{g,HSE}$	$E_{g,GW}$	$E_{g,AFLOW}$	$E_{g,XGBoost}$	$E_{g,SVR}$
GaN	3.20	1.62 (−49%)	3.14 (−2%)	3.32 (+4%)	1.85 (−42%)	3.26 (+2%)	4.45 (+39%)
CdTe	1.60	0.62 (−61%)	1.52 (−3%)	1.76 (+12%)	0.67 (−57%)	0.88 (−45%)	1.43 (−9%)
LiF	14.20	9.2 (−35%)	11.47 (−19%)	15.1 (+6%)	8.27 (−42%)	10.51 (−26%)	9.87 (−30%)
TiO ₂	3.42	2.13 (−37%)	3.67 (+7%)	3.73 (+9%)	2.09 (−38%)	3.24 (−5%)	3.99 (+16%)
CuSbS ₂	1.38	0.9 (−35%)	1.69 (+22%)	1.1 (−20%)	0.79 (−42%)	1.26 (−9%)	1.39 (+1%)
ZnS	3.91	2.07 (−47%)	3.49 (−11%)	4.15 (+6%)	2.6 (−33%)	3.05 (−22%)	3.12 (−20%)
Cu ₂ ZnSnS ₄	1.60	0.28 (−83%)	0.09 (−94%)	1.64 (+3%)	N/A	1.91 (+19%)	1.75 (+9%)
PbTe	0.19	0 (−100%)	0.19 (0%)	0.26 (+36%)	0 (−100%)	0.19 (+1%)	0.2 (+5%)
GaAs	1.52	0.19 (−86%)	1.12 (−26%)	1.52 (0%)	0.24 (−84%)	1.21 (−20%)	1.28 (−15%)
ZnO	3.44	0.67 (−81%)	2.49 (−28%)	3.2 (−7%)	1.87 (−46%)	2.74 (−20%)	3.41 (−1%)
MAD		1.65	0.67	0.22	1.60	0.695	0.75
RMSE		2.1	1.0	0.33	2.25	1.251	1.46

Percentage for each calculated value is with respect to the experimentally measured E_g . The mean absolute deviation (MAD) and root-mean-square error (RMSE) are also shown compared with experimental band gap. The band-gap units are all in electronvolts.

As we have seen the evaluation RMSE for SVR (0.450) is lower than for XGBoost (0.630) in Table 3, suggesting SVR should have perform better in predicting the band gap of unseen composition. However in table 4 XGBoost achieve lower MAD (0.695) and RSME (1.251) as compared to SVR Model (0.75) and (1.46). What this tells us that, lower error on your cross-validation (or training) folds doesnt always translate into lower error on truly unseen data. That gap arises because SVR, with its kernel-based fit, can be more prone to over-fitting the idiosyncrasies of the training dataso it looks great at an overview but then stumbles when given new

composition. XGBoosts gradientboosted trees, by contrast, usually include stronger regularization and an ensemble averaging effect that often yields better generalization to out-of-sample compounds. One thing to note that Machine learning model (XGBoost) does the calculating bandgap task with much lower cost and time as compared to DFT method.

5 Discussion

Aspects to be discussed:

- **Errors, Omissions:** There were few error in the code written by chatgpt.
 - (a) **major issues:** It did not evenly distributed the metals and non-metals in the train and test dataset, even when it was mentioned in the prompt leading to reduce classification accuracy.
 - (b) **minor issues:** It used wrong names for columns in the dataset leading to error.
- **Completeness, Short term memory capacity:** ChatGPT generally performs well on short and focused prompts. However, It forgets to integrate few parts of a long prompt, particularly in code or when asked to add this on top of what you gave earlier.
- **Reproducibility** Reproducibility is not guaranteed. Repeating the same prompt often yields similar but not identical results. Sometimes generated code may use different parameters.
- **Hallucinations** – Sometimes in code generation Chatgpt add libraries or code that has no actual purpose.
- **Learned Lessons:** When using Chatgpt for code generation, it becomes necessary to verify the result. The code I generated from Chatgpt was running fine. However, when I compared the result of the code that I have written myself with the result of Chatgpt’s code, I found lower classification accuracy in Chatgpt’s code.

6 Conclusion

We have presented a twostage machinelearning workflow based on XGBoost that classifies composition as metals or nonmetals and predicts the experimental band gap E_g for those classified as nonmetals. By featurizing 4 604 compositions from the Matbench experimental bandgap dataset using 132 compositional descriptors, our classifier achieved an accuracy of 93.8 % (ROCAUC = 0.983), and the regressor achieved an RMSE of 0.630 eV, $R^2 = 0.809$, and MAD of 0.695 eV on heldout data.

When benchmarked against the SVR model of Zhuo *et al.* [4], our XGBoost regressor showed stronger generalization: it yielded a lower MAD (0.695 eV vs. 0.750 eV) and RMSE (1.251 eV vs. 1.460 eV) on ten unseen test compounds. This improved outofsample performance likely stems from XGBoosts ensemble regularization and robustness to overfitting, in contrast to the kernelbased SVRs sensitivity to the idiosyncrasies of the training distribution.

An advantage of using Machine Learning models for predicting band gap is they are cost efficient and quick as compared to DFT method, making them wellsuited for materials screening.

There are now better ways available of predicting Band Gap that uses Graphical Neural Networks (GNN) incorporating the structural parameter of the compound/composition. Another way to improve the accuracy is by using more experimental band gap values for training so that machine learning model have more data available to find patterns.

7 References

References

- [1] Safa O. Kasap. *Principles of Electronic Materials and Devices*. McGraw-Hill, New York, 3rd edition, 2006.
- [2] Y. Liu, T. Zhao, W. Ju, et al. Materials discovery and design using machine learning. *J. Materiomics*, 3(3):159–177, 2017.
- [3] Anubhav Jain, Logan Ward, Alex Dunn, et al. Matbench v0.1: matbench experimental band gap dataset, 2021. Accessed via Materials Project Leaderboard.
- [4] Y. Zhuo, A.M. Tehrani, and J. Brgoch. Predicting the band gaps of inorganic solids by machine learning. *J. Phys. Chem. Lett.*, 9:1668–1673, 2018.